

Frequent Itemset Mining on the IMDB Top 1000

A-Priori and PCY Compared on Actor Co-occurrence

Gianmarco Costanzo (34873A)

September 2026

Abstract

This exam task consist in using market-basket analysis algorithms explained during the lessons on the IMDB Top 1000 dataset. Every single movie is a basket and my analysis is pecifically focused on the actors columns. I implemented from scratch two of the techniques presented in the course: the A-Priori algorithm and the PCY algorithm. Both were run with support threshold $s = 3$, naturally chosen on the observed density and not on the textbook referenced threshold, which on this dataset would produce 0 association from the doubleton. PCY works better reducing the candidate pairs to be counted from 36,585 to 33 and the running time from 10.238 to 0.017 seconds. The final section is about the structural limitation of the dataset: since support is an absolute count, and the collection is heavily skewed towards recent decades. We can observe a probable correlation between the mean, internet and specifically online rating, and the movies contemporary to that invention and capillar growth towards normal users.

I. Dataset and Basket Construction

1 Dataset Description

The IMDB Top 1000 dataset consist of 1,000 movies described in 16 columns, for years from 1920 to 2020, with IMDB ratings from 7.6 to 9.3 and mean of 7.95. The narrow rating range is a direct consequence of the selection criterion: the collection is not a sample of cinema, it is the upper tail! The four actor columns are the principal item of my analysis and fro what I verify in the module 4 of Python code their structure do not present any missing Value. O n the other hand i found them concentrated in columns the analysis never touches: Certificate (101), Meta_score (157) and Gross (169). One data-quality error was “manually” corrected, The row of *Apollo 13* had the value "PG" in the Released_Year.

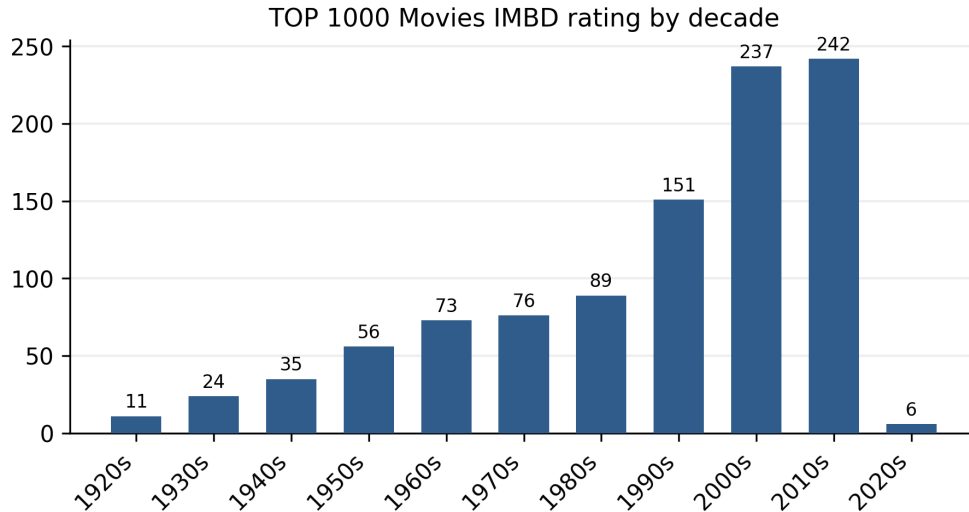


Figure 1: Almost two thirds of the collection comes from the three decades between 1990 and 2019, a skew that becomes relevant in Section IV related in my opinion with the enlargement of Internet instruments exactly in that period

2 Basket Construction

For the construction of compact baskets sets, I transform each csv row that represents an actor into a frozenset, happens based on two different reasons. First of all, a set is an object in which the order of the items and their repetitions are not relevant, as in a normal real grocery basket. Secondly, the itemsets have to be used as dictionary keys during the counting phase and Python only takes

hashable objects for this functionality. An effect of this representation can be seen in the data, four baskets contain three actors rather than the usual four because the director is also one of the actors. This repetition of item is clearly not counted for the structure of set explained before. The final collection has 1,000 baskets over 2,709 distinct actors.

II. Method

1 Support Threshold

Support threshold is the main hyperparameter of Market-Basket-Analysis, and it defines the value for which an item or an itemset is considered *Frequent*. According to the section 6.1.4 in the reference textbook “Mining of Massive Datasets” by A. Rajaraman and J. Ullman, the proposed value for the support threshold is approximately 1% of the total number of baskets. For 1,000 baskets a standard threshold value is $s = 10$ but with this setting I’ll obtain only 9 frequent singleton and not a single frequent pair. The threshold was therefore calibrated on the observed density. With $s = 3$, that is 0.3% of the baskets, I obtain 271 frequent actors out of 2,709 t, exactly 10% of them, and 25 frequent pairs are found. Moving to $s = 4$ the pairs collapse from 25 to 5, which would leave too little to analyse.

2 A-Priori

The A-priori Algorithm is the classical data mining technique to identify frequent items. It starts by looking through all the data to count how many times each single item appears. These single items are called 1-Item-Sets. Then, each further iteration at the k -th level produces the candidate itemsets from the frequent itemsets of the $(k-1)$ -th level using the procedure of join and prune. In this procedure, two $(k-1)$ -itemsets are joined if their union results in a set of k number of items, which happens only if the two $(k-1)$ -itemsets differ by only one item. The generate candidate is discarded immediately if any of its $(k-1)$ -subsets is not frequent. This is the concept of monotonicity, and it avoids paying a full scan of the data for a candidate that cannot possibly be frequent. The candidates that survive are then counted in a single pass over the baskets, and the loop stops when a level produces no candidate at all.

3 PCY

In our case, with the tuned support threshold, we pass from 2709 different actors to 271 frequent singletons in a first passage, and from 271 singletons to 25 frequent pairs in a second one. Considering the structure of the A-priori algorithm, in this second step we have to generate $\binom{271}{2} = 36,585$ potential candidates from the frequent singletons to obtain just 25 frequent pairs. PCY Algorithm exploits exactly the level between singleton and creation of pairs. During the first pass, while single items are being counted, PCY also hashes every pair present in each basket into a table of NUM_BUCKETS counters. On the second pass, a pair becomes a candidate only if both its items are frequent and it hashes to a bucket whose counter reached the threshold. A bucket below the threshold cannot contain any frequent pair, so all the pairs mapped into it are discarded at once, without ever being counted individually. The converse does not hold: a crowded bucket may be the result of several distinct infrequent pairs colliding, which is why PCY produces false positives and a real counting pass is still needed on the surviving candidates. The bucket table was set to 70,000 counters.

III. Results

1 A-Priori by Level

Table 1 reports the behaviour of A-Priori level by level. The algorithm found 299 frequent itemsets in total in 5.1 seconds and terminated at $k = 4$, where the three frequent triples produced no candidate: they correspond to three different franchises with no actor in common, so their unions always have six elements and never four.

Table 1: A-Priori by level, $s = 3$. Selectivity is the share of candidates that turn out to be frequent.

k	Candidates	Frequent	Selectivity
1	2,709	271	10.00%
2	36,585	25	0.07%
3	4	3	75.00%
4	0	—	—

The selectivity column is the argument to implement PCY as I discussed in its proper subsection. At level 2 in the A-Priori the more than 99.9% of the work is spent on itemsets that will be discarded.

At level 3, where the pruning step finally becomes effective, three candidates out of four survive.

2 A-Priori versus PCY

The two algorithms return exactly the same 25 frequent pairs, and the notebook asserts this equality explicitly. This is expected, since PCY is an exact algorithm and not an approximation, and it is used here as a correctness check on both implementations.

Table 2: Comparison on the frequent-pairs step, $s = 3$, 70,000 buckets.

	A-Priori	PCY
Candidate pairs to count	36,585	33
Frequent pairs found	25	25
Frequent buckets	—	33
Running time (s)	10.238	0.017

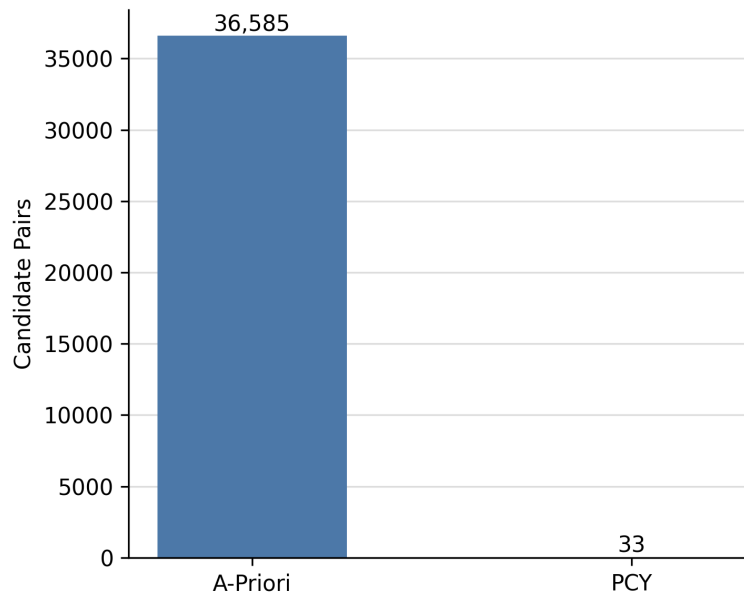


Figure 2: Candidate pairs that each algorithm has to count in the second pass.

To explain the distinctions between the algorithms correctly, it is important to mention that the reduction is not attributable directly to the hash filter. The second pass of PCY enumerates the pairs

by walking the baskets, it only ever considers pairs that actually co-occur in at least one movie, which are 622, while A-Priori generates all the combinations of frequent items. The hash filter is responsible for the step from 622 to 33. The point of the comparison is the number of candidates, which is what would translate into memory and time on a collection of millions of baskets.

3 Frequent Itemsets and Rules

In this subsection I'll give an interpretation of the obtained results, using a very basic external context knowledge. Frequent itemsets are dominated by franchises with a stable cast. The strongest pair is the three combinations of the Harry Potter trio (support 6, 5 and 5). The three frequent triples are the Harry Potter trio (5), the Lord of the Rings group of Elijah Wood, Ian McKellen and Orlando Bloom (3), and the original Star Wars cast of Carrie Fisher, Harrison Ford and Mark Hamill (3). From the frequent itemsets, 39 association rules with confidence at or above 0.5 were extracted. Several reach a confidence of 1, every movie of Tim Allen in the collection also features Tom Hanks, which is the Toy Story series, and the same happens within the Harry Potter and Lord of the Rings groups. Confidence is asymmetric and this is where the measure becomes informative: the rule from Tim Allen to Tom Hanks has confidence 1, while the reverse holds in only 4 of the 14 movies of Tom Hanks, that is 0.29. The support of the pair is identical in both directions, the reading is not.

IV. Limitations

The itemsets found above are correct but they should not be read as a statement about cinema, and the reason is methodological rather than cinematographic. As Figure 1 shows, the collection is heavily skewed, 63% of the movies were released between 1990 and 2019. Since support is an absolute count, a pair of actors needs a certain number of movies in the collection to reach the threshold, and decades represented by a handful of titles have almost no chance of producing one. Running the same search separately on each decade, with the same threshold, the correlation between the number of movies of a decade and the number of frequent pairs found in it is 0.81. In other words, the dominance of recent franchises is largely a sampling artefact of an absolute threshold applied to an unbalanced collection. A second, related bias concerns geography. The fifteen most frequent actors are all American or British, and the first non-anglophone name appears in sixteenth position. The mechanism is the same: accumulating support requires recurring in many movies of the collection, and world cinema is represented here by isolated masterpieces from many different countries, each with a different cast, while Hollywood and Bollywood have a star system

that produces recurrence. The dataset contains no nationality or language column, so this bias can be observed but not measured without joining an external source.

V. Conclusion

The two algorithms behave exactly as the theory predicts. They return the same frequent pairs, which validates both implementations, and PCY removes the bottleneck of A-Priori precisely where it is structurally unavoidable, the second level, reducing the pairs to be counted by roughly three orders of magnitude. Beyond the numerical result, the methodological lesson concerns the difference between support and confidence. Support is an absolute count, and as such it rewards whichever subgroup of the data has more opportunities to appear: this is what makes recent franchises dominate the results, and what hides the recurring casts of the older decades. Choosing the wrong measure for the question being asked, or importing a threshold calibrated on a different kind of data, produces results that are formally correct and substantially misleading. On a dataset this small the consequence is a few missing pairs; on the massive collections these algorithms are designed for, the same reasoning governs which patterns are visible at all.